# Pogrammable Networks
# Project 1

April 2026

## 1 Introduction

In distributed machine learning systems, a set of worker nodes collaboratively train a model by periodically exchanging updates (e.g., gradients or model parameters). A common communication pattern involves a many-to-one aggregation phase, where multiple workers simultaneously send data toward a single aggregation node (collector).

This communication pattern generates a highly synchronized many-to-one traffic demand, commonly referred to as *incast*. During this phase, a large number of flows are directed toward the same destination, traffic is generated in bursts, and the network experiences sudden load concentration.

We consider a data center network with a multi-rooted (leaf-spine) topology, providing multiple equal-cost paths between workers and collectors. An SDN controller has a global view of the network and can dynamically influence forwarding decisions.

The objective is not to eliminate traffic convergence, which is intrinsic to the application, but to efficiently coordinate flows during aggregation phases in order to improve resource utilization and reduce congestion-induced inefficiencies.

## 2 System Model

We consider a set of training procedures $v \in \mathcal{V}$. Each training procedure is associated with:

- a set of worker nodes
$$\{w_1^v, w_2^v, \ldots, w_{K_v}^v\}$$
- a collector node $a_v$, identified by $(IP_v, Port_v)$

Traffic is generated in rounds. During each round, all workers participating in training $v$ transmit data toward the collector $a_v$, giving rise to a synchronized many-to-one communication pattern.

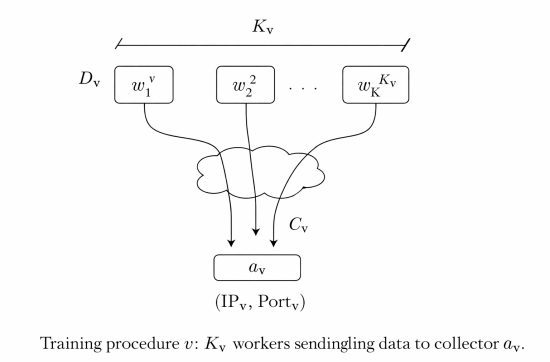


Figure 1: Training procedure $v$: $K_v$ workers sending data to collector $a_v$.

Each worker sends a fixed amount of data $D_v$ per round. Transmissions occur periodically with period $T_v$ and phase $\phi_v$. While workers within the same training are approximately synchronized, different training procedures are not aligned in time and may overlap.

For simplicity, we assume that the temporal behavior $(T_v, \phi_v)$ is constant over time. Although in practice this may depend on network conditions, this assumption allows the controller to infer a stable traffic pattern.

From a network perspective, each transmission corresponds to a TCP flow directed toward $(IP_v, Port_v)$. The controller knows the set of collectors and the network topology, modeled as a graph $G(\mathcal{N}, \mathcal{L})$ with link capacities $C_\ell$, but it does not know a priori:

$$K_v, \quad D_v, \quad T_v, \quad \phi_v$$

These parameters must be inferred from observed traffic.

The total traffic generated by training $v$ in one round is:

$$K_v \cdot D_v$$

This demand is concentrated in short intervals, generating bursty incast traffic. When multiple training procedures overlap, the network experiences the superposition of multiple such events.

# 3 Problem Statement

Consider a training procedure $v$ with $K_v$ workers, each transmitting $D_v$ data units toward collector $a_v$. All flows share an effective aggregation capacity $C_v$, corresponding to the bottleneck link toward the collector.

Under ideal conditions, this is the only bottleneck. Assuming fair sharing, each flow achieves throughput $C_v/K_v$, and the completion time of a round is:

$$T_v = \frac{K_v D_v}{C_v}$$

This represents a lower bound on the completion time.

However, this ideal behavior is achieved only if traffic is well distributed across the network. If multiple flows are routed along overlapping paths, congestion may arise at intermediate links before reaching the collector. In this case, the effective bottleneck shifts from the collector link to upstream links.

This leads to reduced throughput, increased queueing, and potential packet losses, ultimately increasing completion time beyond the ideal bound.

This issue is particularly relevant in leaf-spine topologies, where multiple equal-cost paths exist. Poor routing decisions may concentrate traffic on a subset of links, creating avoidable congestion hotspots while leaving other paths underutilized.

The role of the SDN controller is therefore to ensure that the collector link remains the dominant bottleneck by properly distributing traffic across the network.

# 4 Project Requirements

Students are required to design and implement an SDN application that monitors, characterizes, and controls traffic generated by training procedures.

The application operates at the controller level and installs forwarding rules to influence traffic distribution.

The system must include the following components:

- **Worker Discovery** The controller must identify the set of workers participating in each training procedure $v$ by observing flows directed toward $(IP_v, Port_v)$.
- **Traffic Characterization** For each training $v$, the controller must estimate:
$$K_v, \quad D_v, \quad T_v, \quad \phi_v$$
- **Traffic Control** Based on the inferred parameters, the controller must implement a policy that distributes flows across the network in order to:
  - avoid congestion at intermediate links
  - balance load across available paths
  - approach the ideal completion time

The emphasis is on simple, effective strategies that combine traffic inference with network control.